

Module 3 — MongoDB Data Models

Time: 1 hour · Covers Practical 1 · CO4

Practical 1: Write a program to implement MongoDB data models.

Part A — What is MongoDB? (concept, ~15 min)

MongoDB is a **NoSQL, document database**. Instead of tables with rows and columns (like SQL), it stores **documents** that look like JSON.

SQL term	MongoDB term
database	database
table	collection
row	document
column	field

A single `user` document:

```
{
  "_id": "652f...",           // MongoDB adds a unique _id automatically
  "name": "Ada Lovelace",
  "email": "ada@example.com",
  "age": 36,
  "hobbies": ["math", "poetry"], // arrays are fine
  "address": { "city": "London" } // nested objects are fine
}
```

Documents are **flexible**: two documents in the same collection can have different fields. That flexibility is powerful but dangerous — without rules your data becomes inconsistent. That is why we use **Mongoose**.

Part B — Mongoose: schemas & models (~20 min)

Mongoose is a library that puts *structure* on top of MongoDB. Two key ideas:

- **Schema** — the shape and rules of a document (which fields, their types, what is required, defaults, validation).
- **Model** — a constructor built from a schema. You use the model to create, read, update, and delete documents. A model maps to one **collection**.

```
const mongoose = require("mongoose");

// 1) Define the SCHEMA: the shape + rules.
const userSchema = new mongoose.Schema({
  name: { type: String, required: true },           // must be present
  email: { type: String, required: true, unique: true }, // no duplicates
  age: { type: Number, min: 0, max: 120 },         // range validation
});
```

```

    role: { type: String, enum: ["student", "admin"], default: "student" },
    createdAt: { type: Date, default: Date.now },          // auto timestamp
  });

// 2) Build the MODEL from the schema. "User" -> "users" collection.
const User = mongoose.model("User", userSchema);

```

Now `User` validates every document for you: a missing `name`, a duplicate `email`, or an `age` of 999 will be rejected before it reaches the database.

Part C — Modeling relationships (~15 min)

Real data is connected: a **post** has an **author**, an **order** has **items**. MongoDB offers two ways to model this.

1. Referencing (store the other document's `_id`)

Best when the related data is large or shared.

```

const postSchema = new mongoose.Schema({
  title: { type: String, required: true },
  // store the author's _id and tell Mongoose which model it points to
  author: { type: mongoose.Schema.Types.ObjectId, ref: "User" },
});

```

Later you can `.populate("author")` to pull in the full user document.

2. Embedding (put the sub-document inside)

Best when the data is small and always read together.

```

const orderSchema = new mongoose.Schema({
  customer: String,
  // an array of embedded sub-documents – no separate collection
  items: [
    {
      product: String,
      quantity: { type: Number, default: 1 },
      price: Number,
    },
  ],
});

```

Rule of thumb: embed when data is "owned" and read together (order items); reference when data is shared or grows without bound (a user's posts).

The complete, commented models are in [code/models.js](#).

Part D — Run it (~10 min)

The demo connects to MongoDB, defines the models, creates a few documents, and prints them.

```
cd 03-mongodb-models
npm install
cp .env.example .env      # then edit .env with YOUR MongoDB connection string
node code/demo.js
```

If you do not have MongoDB running locally, put your **Atlas** connection string in `.env` (see Module 0).

See [code/demo.js](#) — it shows validation catching a bad document, referencing with `.populate()`, and embedding.

Summary

- MongoDB stores flexible **documents** in **collections**.
- **Mongoose Schema** = the rules; **Model** = the tool to use them.
- Model relationships by **referencing** (`ObjectId` + `ref`) or **embedding**.
- Schemas give you validation, defaults, and safety on flexible data.

Now do [practice.md](#) .